INTRO TO SOFTWARE ENGINEERING · 2569 · MILESTONE 1

เลือกปัญหาให้ถูก ก่อนจะสร้างของที่ไม่มีใครใช้

คู่มือสำหรับนักศึกษา เพื่อหา “หัวข้อโปรเจกต์” ที่มีคนอยากใช้จริง จากปัญหาใกล้ตัว สู่ปัญหาที่มี value และตลาด แล้วลงมือสร้างจริงด้วย AI

ประโยคเดียวที่ต้องจำ วิชานี้ไม่ได้ให้คะแนน “แอปที่สวย” แต่ให้คะแนน **ความเข้าใจปัญหา** ถ้าเข้าใจปัญหาลึกพอ product จะมีคนใช้เอง ถ้าเข้าใจตื้น ต่อให้โค้ดสวยแค่ไหน ก็ไม่มีใครเปิดครั้งที่สอง

ในคู่มือนี้ · WHAT'S INSIDE

01 หลงรักปัญหา ไม่ใช่โซลูชัน

02 กับดัก “ปัญหาใกล้ตัวเกินไป”

03 บททดสอบ platform แก้ได้จริงไหม

04 Root Cause ด้วย 5 Whys

05 TAM / SAM / SOM แบบง่าย

06 เครื่องคิดเลข TAM ลองเลย

07 ขยายมุมมอง มองให้กว้างขึ้น

08 Reframe ตัวอย่างจริง

09 Scorecard วัดคุณภาพปัญหา

10 ตัดสินใจ GO / REFRAME / DROP

11 การบ้าน Problem Brief

12 ไปค้นคว้าที่ไหนต่อ

13 ก้าวต่อไป แนวคิด AI SDLC

14 Starter Kit ตัวอย่างของอาจารย์

15 M1 ส่งอะไร + เกณฑ์ให้คะแนน

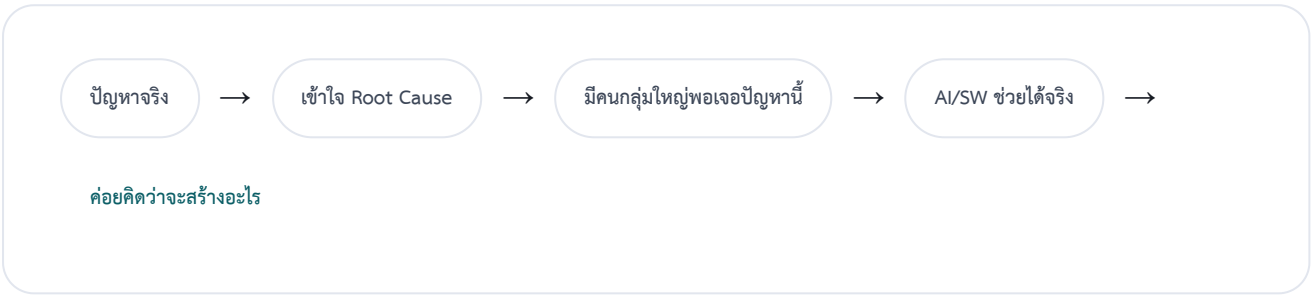
01 · MINDSET / กรอบความคิด

“หลงรักปัญหา ไม่ใช่หลงรักโซลูชัน”

Fall in love with the problem, not the solution — คำเตือนคลาสสิกของคนทำ product ทุกคน

หลายทีมมักเริ่มจาก “อยากทำแอป...” หรือ “อยากทำ platform...” แล้วค่อยไปหาปัญหามาใส่ทีหลัง นั่นคือการ หลงรักโซลูชัน ปัญหาคือ ถ้าเราผูกใจกับ “แอป” ไปแล้ว เราจะมองไม่เห็นว่าปัญหาที่เลือกมัน เล็ก ตื้น หรือแก้ด้วยซอฟต์แวร์ไม่ได้จริง เพราะเราแค่อยากหาข้ออ้างให้ได้สร้างแอป

ลำดับที่ถูกต้องกลับกัน



ทำไมเรื่องนี้สำคัญกับคะแนนของคุณ

รูบริกของวิชานี้ให้คะแนน Coherence (ความต่อเนื่อง) คือ Problem → Requirement → Design → Feature → Test ต้องเป็น “เรื่องเดียวกัน” ถ้าปัญหาตั้งต้นอ่อน ทุกอย่างที่ต่อจากมันก็อ่อนตามหมด และ M4+M5 จะถูก cap ที่ 50% ถ้าเส้นด้ายขาด เลือกปัญหาดีจึงเท่ากับลงทุนให้คะแนนทั้งเทอม

ตัวอย่างสั้น ๆ

โซลูชันก่อน “อยากทำแอปจดไดอารี่ด้วย AI” แล้วต้องไปหาว่าใครเดือดร้อนเรื่องจดไดอารี่ (หายาก) ปัญหาก่อน “นักศึกษาปี 1 หลายคนเครียดจนนอนไม่หลับแต่ไม่กล้าไปพบนักจิตวิทยา” พอเข้าใจปัญหานี้ลึก ๆ ไดอารี่ AI อาจเป็นแค่ หนึ่ง ทางออก และอาจมีทางที่ดีกว่าโผล่ขึ้นมาเอง

กับดัก “ปัญหาใกล้ตัวเกินไป”

ปัญหาใกล้ตัวไม่ได้ผิด มันเป็นจุดเริ่มที่ดีเพราะคุณ “รู้สึก” ได้จริง
แต่หลายปัญหาใกล้ตัว ไม่ได้แก้ด้วย platform และ ตลาดเล็กเกินกว่าจะมีใครมา
ใช้ต่อ ลองดูสองตัวอย่างที่เจอบ่อยกันตรง ๆ

ตัวอย่าง A — “แอปหาหอพักแถวมหาลัย”

ทำไมมันไม่ใช่ปัญหาซอฟต์แวร์

- ปัญหาจริงคือ ห้องมีจำกัด ราคาแพง เจ้าของไม่โปร่งใส เป็นปัญหา supply & trust ไม่ใช่ปัญหา “หาข้อมูลไม่เจอ”
- แอปไม่ได้สร้างหอเพิ่ม ไม่ได้ทำให้ถูกลง ไม่ได้บังคับเจ้าของให้ซื่อสัตย์
- Facebook group เพจหอ หรือรุ่นพี่บอกต่อ ทำงานนี้ “ดีพอ” อยู่แล้ว ต้องแข่งกับของฟรีที่มีอยู่

ปัญหาเชิงตลาด ทำไมไม่มีคนใช้ต่อ

- ใช้ครั้งเดียวต่อปี หรือต่อสี่ปี ความถี่ต่ำมาก (low frequency)
- ผู้ใช้แค่ “นักศึกษามหาลัยเดียว” ตลาดหลักพันคน ไม่ใช่หลักล้าน
- Two-sided cold start ถ้าไม่มีเจ้าของหอมาลง ก็ไม่มีนักศึกษาเข้า ถ้าไม่มีนักศึกษา เจ้าของก็ไม่ลง เป็นไก่กับไข่

ตัวอย่าง B — “แอปจองรถเช่าแถวมหาลัย”

แก่นของปัญหาไม่ได้อยู่ที่ “การจอง”

- ปัญหาจริงคือ ความน่าเชื่อถือ ประกัน และความรับผิดถ้ารถเสียหรือชน เป็นเรื่องกฎหมายและความเสี่ยง ไม่ใช่ปุ่มจอง
- ร้านเช่ามีไม่กี่เจ้า คุยผ่าน LINE ก็จบ แอปไม่ได้ลดแรงเสียดทานที่แท้จริง
- เรื่องเงิน ประกัน มัดจำ เกินขอบเขตที่โปรเจกต์นักศึกษาจะรับผิดชอบได้

สัญญาณเตือนร่วม

- เป็น marketplace ที่ต้องมีทั้งฝั่งซัพพลายและดีมานด์ ยากสุดสำหรับทีมเล็ก
- “value” ที่แท้จริงอยู่ที่ ของจริง เช่น รถหรือห้อง ไม่ใช่ที่ซอฟต์แวร์ ซอฟต์แวร์จึงเป็นแค่ 5% ของคุณค่า
- ถ้าตอบไม่ได้ว่า “ทำไมคนถึงเลิกใช้ Facebook หรือ LINE มาใช้ของเรา” นั่นคือสัญญาณอันตราย

ประเด็นสำคัญ

ปัญหาเหล่านี้ เป็นปัญหาจริง แต่เป็นปัญหาที่แก้ด้วย ซัพพลาย ความไว้ใจ กฎหมาย หรือเงิน ไม่ใช่แก้ด้วย “หน้าจอแอป” งานของ software engineer คือแยกให้ออกว่า ส่วนไหนของปัญหาเป็นปัญหาข้อมูลหรือกระบวนการ ที่ซอฟต์แวร์ช่วยได้จริง ก่อนจะลงมือสร้าง

บททดสอบ “platform แก้ปัญหานี้ได้จริงไหม”

ก่อนจะรักไอเดียไหน ให้ปัญหานั้นเดินผ่าน 5 คำถามนี้ก่อน ถ้าติดหลายข้อ แปลว่ายังไม่ใช่ปัญหาซอฟต์แวร์ที่ดี

1 **แก่นของปัญหาเป็นเรื่อง “ข้อมูล การตัดสินใจ การทำงานซ้ำ” หรือเป็นเรื่อง “ของจริงที่ขาด”**

ซอฟต์แวร์เก่งเรื่องจัดข้อมูล เชื่อมคน ทำงานซ้ำอัตโนมัติ วิเคราะห์และแนะนำ แต่มันไม่เก่งเรื่องเพิ่มห้องพัก เพิ่มรถ หรือเปลี่ยนใจคน

red flag ถ้าคุณค่าหลักอยู่ที่ “ของจริง” เช่น ห้องหรือรถ ให้ถือเป็นสัญญาณอันตราย ว่านี่อาจไม่ใช่ปัญหาซอฟต์แวร์

2 **มีปัญหานี้บ่อยแค่ไหน**

ปัญหาที่ใช้เดือนละหลายครั้งดีกว่าปีละครั้ง ความถี่สูงแปลว่าคนกลับมาใช้ และ product มีชีวิต

3 **ตอนนี้เขาแก้ปัญหานี้ยังไง แล้วมัน “แย่พอ” ไหม**

ถ้าวิธีเดิม เช่น Excel LINE จดใส่กระดาษ หรือ Facebook group “ดีพอแล้ว” คนจะไม่เปลี่ยน ต้องเจ็บพอที่จะยอมเปลี่ยน (a “hair-on-fire” problem)

4 **ต้องมีผู้ใช้สองฝั่งไหม (marketplace)**

ถ้าใช่ นั่นคือ cold-start problem ยากมากสำหรับทีมเล็กที่ไม่มีงบซื้อผู้ใช้ เลือกปัญหาที่ให้คุณค่าได้ตั้งแต่ผู้ใช้คนแรก (single-player value) จะปลอดภัยกว่า

5 **AI/SW ทำให้ดีขึ้น “อย่างมีนัยสำคัญ” จริงไหม**

ไม่ใช่แค่ “ย้ายของเดิมมาไว้ในแอป” ต้องมีอย่างน้อย 1 จุดที่ AI หรือ ตรรกะ ทำสิ่งที่มนุษย์ทำเองไม่ไหวหรือช้ากว่ามาก

กฎง่าย ๆ

ถ้าเอา “แอปหรือซอฟต์แวร์” ออกจากไอเดียแล้ว ปัญหายังแก้ได้เกือบหมดด้วยการโทร แชท หรือนัดเจอ แปลว่าซอฟต์แวร์ไม่ใช่พระเอก ให้หา **ปัญหาที่ถ้าไม่มีซอฟต์แวร์จะแก้ไม่ได้เลยหรือแก้ได้แย่มาก**

ขุดหา Root Cause ด้วย “5 Whys”

ปัญหาที่คุณ “เห็น” มักเป็นแค่ อาการ (symptom) ถามว่า “ทำไม” ซ้ำ ๆ 5 ครั้งจะเจอสาเหตุจริง และบ่อยครั้ง สาเหตุจริงพาเราไปเจอปัญหาที่ใหญ่และมีค่ากว่า

สาธิตกับตัวอย่าง A หอพัก

W1 ทำไมหาหอพักยาก

เพราะข้อมูลกระจัดกระจาย ตามเพจและกลุ่มต่าง ๆ

W2 ทำไมข้อมูลกระจัดกระจาย

เพราะไม่มีใครรวบรวม และเจ้าของหอไม่อัปเดตห้องว่าง

W3 ทำไมเจ้าของไม่อัปเดต

เพราะไม่มีแรงจูงใจ ห้องเต็มอยู่แล้ว หรือใช้ปากต่อปากก็พอ

W4 ทำไมนักศึกษายอมทน

เพราะย้ายหอปีละครั้ง เจ็บช่วงสั้น ๆ แล้วก็ลืม

W5 แล้วอะไรคือความเจ็บที่ “แท้จริงและซ้ำ ๆ”

ไม่ใช่ “หาห้อง” แต่คือ ไม่รู้ว่าหอไหนน่าเชื่อถือ รีวิวจริงหรือปลอม จะโดนโกงมัดจำไหม

Root cause ที่เจอคือปัญหา “ความไว้ใจในการเช่าที่พักระยะสั้น” ซึ่งเป็นปัญหาระดับประเทศ ไม่ใช่แค่หน้ามอ นี่คือปัญหาที่ใหญ่ขึ้น มีค่ามากขึ้น และซอฟต์แวร์ เช่น รีวิวที่ยืนยันตัวตน หรือ ตรวจจับรีวิวปลอมด้วย AI ช่วยได้จริง

บทเรียน

การขุด root cause ไม่ได้แค่ช่วยให้เข้าใจปัญหาลึกขึ้น แต่มักจะ “ยกระดับ” ปัญหา จากเรื่องเล็กหน้ามอ ไปเป็นปัญหาที่กว้างและมีตลาดใหญ่กว่า โดยที่ยังเป็นเรื่องที่คุณ “อิน” อยู่ นี่คือสะพานสู่หัวข้อถัดไป

05 · MARKET SIZE / ขนาดตลาด

TAM / SAM / SOM ขนาดตลาดแบบเข้าใจใน 3 นาที

“value” ที่พูดถึงกันในวิชานี้ วัดเป็นตัวเลขได้ ด้วยกรอบง่าย ๆ ที่ startup ทั้งโลกใช้ ไม่ต้องแม่นยำ ขอแค่ “ประมาณบนกระดาษเช็ดปาก (napkin math)” เป็น

TAM

Total Addressable Market

“ตลาดทั้งหมด” คือทุกคนบนโลกหรือในประเทศ ที่มีปัญหานี้ ถ้าเราครองตลาดได้ 100%

คำถาม ใครบ้างที่มีปัญหานี้ทั้งหมด

SAM

Serviceable Available Market

ส่วนที่ product เรา “เข้าถึงได้จริง” จำกัดด้วยภาษา พื้นที่ ช่องทาง แพลตฟอร์ม

คำถาม เราให้บริการใครได้จริงในเทคโนโลยี ภาษา และพื้นที่ของเรา

SOM

Serviceable Obtainable Market

ส่วนที่เรา “คว้าได้จริงใน 1–2 ปีแรก” เมื่อคิดถึงคู่แข่งและกำลังของทีม

คำถาม ปีแรกเราจะได้ผู้ใช้จริงกี่คน

คิดง่าย ๆ เป็นวงกลมซ้อนกัน TAM ⊃ SAM ⊃ SOM เราเริ่มจาก SOM ที่จับได้จริง แต่ต้องมั่นใจว่า TAM ใหญ่พอที่จะ “คุ้มค่าที่จะลงมือ”

ทำไมตัวอย่าง “หอพักหน้ามอ” ถึงสอบตกเรื่องตลาด

ระดับ	หอพักหน้ามอ แบบแคบ	ประมาณจำนวน
TAM	นักศึกษา MFU ที่หาหอต่อปี	~3,000

ระดับ	หอพักหน้ามอ แบบแคบ	ประมาณจำนวน
SAM	ที่ใช้สมาร์ตโฟนและยอมลองแอปใหม่	~2,000
SOM	ที่เราดึงมาได้จริงปีแรก	~200–400

TAM หลักพันบวกความถี่ปีละครั้ง เท่ากับตลาดเล็กมาก ต่อให้ทำสำเร็จ 100% ก็ยังเล็ก นี่คือสัญญาณให้ “มองกว้างขึ้น”

NAPKIN MATH ที่ยอมรับได้

เราไม่ต้องการตัวเลขเป๊ะ เราต้องการให้คุณ เขียนสมมติฐานออกมาเป็นเลข เช่น “คน × ความถี่ × มูลค่าต่อครั้ง” ถ้าเขียนแล้วตัวเลขมันเล็กจนน่าตกใจ ก็ถือว่าดี เพราะคุณเพิ่งประหยัดเวลาทั้งเทอม

06 · TRY IT / ลองเลย

เครื่องคิดเลข TAM แบบ napkin

ใส่สมมติฐานของทีมคุณ แล้วดูว่าตลาดใหญ่พอไหม ปรับตัวเลขเล่นได้ จุดประสงค์คือให้ “รู้สึก” ถึงขนาดตลาด

จำนวนคนที่มีปัญหานี้ (คน) People with the problem	3000
ใช้กี่ครั้งต่อปี ต่อคน Uses per person / year (frequency)	1
มูลค่าต่อการใช้ 1 ครั้ง (บาท) Value you could capture / use (฿)	20
ส่วนที่เข้าถึงได้จริง SAM (%) Reachable share	60
ส่วนที่คว้าได้ปีแรก SOM (% ของ SAM) Obtainable in year 1	15

TAM / ปี	SAM / ปี	SOM / ปี
฿60.0K	฿36.0K	฿5.4K
ตลาดทั้งหมด	เข้าถึงได้	คว้าได้จริง

ตลาดเล็กมาก TAM ต่ำกว่า ฿0.5M ต่อปี ลอง “ปีนบันไดปัญหา” หาเวอร์ชันที่กว้างขึ้น หรือเพิ่มความถี่หรือมูลค่าต่อครั้ง

* ตัวเลขนี้เป็นเครื่องมือ “คิดเร็ว” ไม่ใช่ตัวเลขจริงจากตลาด ใช้เพื่อเทียบไอเดีย A กับ B ว่าอันไหน “ตลาดใหญ่กว่ากันเยอะ ๆ” เท่านั้น

07 · ZOOM OUT / ขยายมุมมอง

มองให้กว้างขึ้น “ladder up” จากปัญหาเล็กสู่ปัญหาใหญ่

ไม่ได้แปลว่าต้องทิ้งปัญหาใกล้ตัว แต่ให้ “ปีนบันได” จากมันขึ้นไป หาปัญหาที่ใหญ่และทั่วไปกว่า แล้วค่อยเลือกขั้นที่ “ใหญ่พอมีตลาด แต่เล็กพอที่ทีมเราทำ MVP ได้”

เทคนิค “ปีนบันไดปัญหา” (Problem Ladder)

ขั้นล่างสุด หาหอพักหน้า MFU

↑ ถามว่า “ปัญหานี้เวอร์ชันทั่วไปคืออะไร”

กว้างขึ้น นักศึกษาทั่วไทยหาที่พักระยะสั้นที่เชื่อถือได้

↑ ถามอีกที

ระดับปัญหาจริง “คนเช่าที่พักไม่รู้ว่าจะเชื่อรีวิวหรือเจ้าของได้แค่ไหน” เป็นปัญหาความไว้ใจ ที่ AI ช่วยตรวจรีวิวปลอมและยืนยันตัวตนได้จริง

สังเกตว่าเราขึ้นบันไดจนเจอ “ชั้นที่ซอฟต์แวร์หรือ AI เป็นพระเอกจริง” และตลาดใหญ่ขึ้นหลายเท่า โดยที่ยังต่อยอดจากสิ่งที่เราอินตั้งแต่แรก

ไปหา “ปัญหาที่มีค่า” จากที่ไหน

อย่ามองแค่ในหอ ในห้องเรียน ปัญหาที่มีคนใช้จริง มักโผล่ในคอมมูนิตี้ที่คนกำลัง “บ่น” เรื่องเดียวกันเป็นพัน ๆ คน

คอมมูนิตี้คนสร้าง (Builder communities)

- Claude / Anthropic community, Discord, r/ClaudeAI — คนเอา AI ไปแก้ปัญหางานจริง เห็น pattern ว่าอะไรที่คน “อยากให้ AI ช่วย”
- Product Hunt — ดูว่าอะไรกำลังมา ปัญหาไหนมีคนทำแล้วบ้าง และช่องว่างที่ยังเหลือ
- Indie Hackers / r/SaaS — คนเล่าว่าปัญหาไหน “มีคนยอมจ่าย”
- Reddit / Pantip — subreddit หรือห้องที่ตรงกลุ่มเป้าหมาย อ่านโพสต์ที่ถูก upvote เยอะ คือปัญหาที่คนอินจริง

ปัญหาระดับประเทศ (National-scale)

ตลาดใหญ่ มี value ต่อสังคม และมักมีข้อมูลให้ AI ทำงาน

- สังคมสูงวัย — ดูแลผู้สูงอายุ สุขภาพ ความเหงา การกินยา
- เกษตร — ราคาผลผลิต โรคพืช ตลาดกลาง
- SME ดิจิทัล — ร้านค้ารายเล็กที่ยังทำบัญชี สต็อก การตลาดด้วยมือ
- การศึกษาและความเหลื่อมล้ำ — การเข้าถึงติวเตอร์ ภาษา ทักษะ
- สุขภาพจิตและการเข้าถึงบริการสุขภาพ ในพื้นที่ห่างไกล

Reframe จากไอเดียอ่อน สู่ไอเดียที่มีคนใช้

สองตัวอย่างเดิม ถ้าปีนบันไดและขุด root cause แล้ว มันกลายเป็นอะไรได้บ้าง

BEFORE — แคบ

“แอปหาหอพักหน้า MFU”

TAM ~3,000 คน ใช้ปีละครั้ง แข่งกับ Facebook ฟรี ซอฟต์แวร์ไม่ใช่พระเอก

AFTER — มี VALUE

“ตัวช่วยตรวจ ‘รีวิวหอหรือห้องเช่าปลอม’ ด้วย AI และยืนยันผู้เช่าจริง”

ปัญหาความไว้ใจ ระดับประเทศ AI ตรวจ pattern รีวิวปลอมได้จริง และเป็น single-player value ใช้ได้แม้เจ้าของไม่ร่วม

BEFORE — แคบ

“แอปจองรถเช่าแถวมอ”

marketplace 2 ฝั่ง เรื่องเงินและประกันเกินสโคป ร้านมีไม่กี่เจ้า

AFTER — มี VALUE

“ผู้ช่วยตรวจสภาพรถก่อนและหลังเช่า ด้วยรูปถ่ายบวก AI จับรอยเพื่อกันข้อพิพาทมัดจำ”

แก้ root cause จริง คือข้อพิพาทและความไว้ใจ AI vision ทำสิ่งที่คนตรวจเองพลาด ใช้ได้กับการเช่าของทุกชนิด ตลาดกว้าง

สังเกต PATTERN การ REFRAME

ทั้งสองเคส เราย้ายจาก “ทำ marketplace หรือ ไดเรกทอรี” ที่ซอฟต์แวร์เป็นตัวประกอบ ไปเป็น “ใช้ AI แก้จุดเจ็บที่แท้จริง คือความไว้ใจหรือข้อพิพาท” ที่ **ซอฟต์แวร์เป็นพระเอก ให้คุณค่าตั้งแต่ผู้ใช้คนแรก และตลาดกว้างขึ้น**

Scorecard ปัญหาของทีมคุณแข็งแรงพอไหม

ให้คะแนนแต่ละข้อ 0–3 โดย 0 คือไม่เลย และ 3 คือใช่เต็ม ๆ เอาไปใช้ก่อนส่ง Charter ที่ M1

1. Real & Specific ปัญหาจริง มีคนที่ระบุชื่อหรือกลุ่มได้เจอจริง คุยกับผู้ใช้จริงได้อย่างน้อย 3 คนก่อน Week 4	0 1 2 3
2. Frequent เกิดบ่อย ระดับสัปดาห์หรือเดือน มากกว่าปีละครั้ง	0 1 2 3
3. Painful วิธีแก้ปัจจุบันแย่พอที่คนอยากเปลี่ยน (hair-on-fire)	0 1 2 3
4. Software-solvable แก่นของปัญหาคือข้อมูล การตัดสินใจ หรืองานซ้ำ ไม่ใช่ของจริงที่ขาด	0 1 2 3
5. AI adds real value มีอย่างน้อย 1 จุดที่ AI หรือ ตรรกะ ทำได้ดีกว่ามนุษย์มาก	0 1 2 3
6. Market big enough (TAM) ถ้าขยายเต็มที่ ตลาดใหญ่กว่า “คนหลักพัน” ระดับจังหวัดหรือประเทศ	0 1 2 3
7. Single-player value ให้คุณค่าได้ตั้งแต่ผู้ใช้คนแรก ไม่ต้องรอทั้งสองฝั่งของ marketplace	0 1 2 3
คะแนนรวม	0 / 21

ตัดสินใจจริง GO / REFRAME / DROP

อ่านมาถึงตรงนี้ คุณมีเครื่องมือครบแล้ว ส่วนนี้คือ “จุดตัดสินใจ” เอาผลจาก litmus test, 5 Whys, TAM และ Scorecard มารวมกันเป็นคำตอบเดียวว่าหัวข้อนี้ ไปต่อ ปรับใหม่ หรือทิ้ง

แผนผังตัดสินใจ (Decision Tree)

ขั้นที่ 1 ปัญหานี้ผ่าน “บททดสอบ 5 ข้อ” จาก section 03 กี่ข้อ

ตกข้อ “ซอฟต์แวร์ไม่ใช่พระเอก” หรือ “เป็น marketplace 2 ฝั่ง”

อย่าเพิ่งทิ้ง ลอง 5 Whys เพื่อหา root cause ที่ซอฟต์แวร์ช่วยได้ ก่อนตัดสิน

ผ่านเกือบหมด

ไปขั้นที่ 2

ขั้นที่ 2 TAM จาก section 06 ใหญ่แค่ไหน

TAM < ฿0.5M/ปี

ตลาดเล็กเกินไป ให้ ปีนบันไดปัญหา จาก section 07 หาเวอร์ชันกว้างขึ้น แล้วคำนวณใหม่

TAM พอหรือใหญ่

ไปขั้นที่ 3

ขั้นที่ 3 Scorecard จาก section 09 ได้กี่คะแนน

0–9 · DROP

ปัญหายังอ่อน เปลี่ยนหัวข้อ หรือ reframe แรง ๆ

10–14 · REFRAME ดูข้อที่ได้คะแนนน้อย แล้วซ่อมจุดนั้นก่อน

15–21 · GO พร้อมทำ Charter และล็อกผู้ใช้จริง 3 คน

NON-NEGOTIABLES 3 ประตูที่ต้องผ่านทั้งหมด

ต่อให้คะแนนรวมสูง แต่ถ้า ตกข้อใดข้อหนึ่ง ต่อไปนี้ ให้ถือว่า “ยังไม่ GO” คือ ข้อ 4 Software-solvable ต้องได้ ≥ 2 ข้อ 5 AI adds real value ต้องได้ ≥ 2 และ ข้อ 1 Real & Specific ต้องได้ ≥ 2 พร้อมระบุ “ผู้ใช้จริง 3 คน” ได้ สามข้อนี้คือหัวใจของวิชา ขาดข้อใดข้อหนึ่ง เส้นด้าย Problem⟶Test จะขาด

เปรียบเทียบ 2–3 ไอเดีย แล้วเลือกอันเดียว

อย่าตัดสินใจจากไอเดียเดียว ให้แต่ละทีมมีตัวเลือกอย่างน้อย 2 ไอเดีย เอามาวางเทียบในตารางนี้ แล้วเลือกอันที่คะแนนสูงสุด และ ผ่านทั้ง 3 ประตูข้างบน จะพิมพ์หรือเขียนมือลงตารางก็ได้

เกณฑ์	ไอเดีย A	ไอเดีย B	ไอเดีย C
Scorecard (0–21)	__ / 21	__ / 21	__ / 21
TAM / ปี (฿)	฿ __	฿ __	฿ __
ผ่าน 3 ประตูไหม	Y / N	Y / N	Y / N
ผู้ใช้จริง 3 คน หาได้จริงไหม	Y / N	Y / N	Y / N
ทีม “อิน” กับปัญหานี้แค่ไหน	__/5	__/5	__/5
คำตัดสิน	GO / REFRAME / DROP	GO / REFRAME / DROP	GO / REFRAME / DROP

ถ้าไม่มีไอเดียไหน GO เลย

นั่นคือ “สัญญาณที่ดี” ไม่ใช่ความล้มเหลว เพราะคุณเพิ่งกันตัวเองจากการทำโปรเจกต์ที่ไม่มีคนใช้ ให้กลับไปที่ section 07 ปีนบันได หรือ section 12 ไปดูปัญหาที่คนบ่นจริง แล้วหาไอเดียใหม่ **เป้าหมายไม่ใช่รีบเลือก แต่คือเลือกให้ถูก**

การบ้าน “Problem Brief” หนึ่งหน้า

ก่อนเสนอหัวข้อในคาบหน้า ให้แต่ละทีมกรอกบรีฟนี้ จะพิมพ์หรือเขียนมือก็ได้ ถ้ากรอกครบและ “ผ่านตัวเอง” ได้ แปลว่าพร้อมทำ

Charter

1. **ปัญหา หนึ่งประโยค** “ใคร ประสบปัญหาอะไร ในสถานการณ์ไหน”

2. **Root cause จาก 5 Whys** สาเหตุที่แท้จริง ไม่ใช่อาการ

3. **ตอนนี้คนแก้ปัญหานี้ยังไง แล้วมันแย่ตรงไหน** วิธีเดิม และทำไมยังไม่ดีพอ

4. **ใครมีปัญหานี้บ้าง และประมาณกี่คน (TAM)** napkin math

5. **ใช้บ่อยแค่ไหน ความถี่** ครั้งต่อสัปดาห์ หรือต่อปี

6. **AI/SW ช่วยตรงไหนที่มนุษย์ทำเองไม่ไหวหรือช้ากว่ามาก** จุดที่ซอฟต์แวร์เป็นพระเอก

7. **ถ้าเอา “แอป” ออก ปัญหายังแก้ได้ไหม เป็นการทดสอบว่าซอฟต์แวร์จำเป็นจริง** ตอบตรง ๆ

8. **หาผู้ใช้จริง 3 คนแรกได้ที่ไหน ก่อน Week 4** ต้องระบุได้ ไม่งั้นไม่ผ่าน Gate 1

“ผู้ใช้จริง (real user)” คือใครได้บ้าง

นักศึกษาส่วนใหญ่เข้าใจผิดว่า real user คือ “คนที่ฉันเดินไปเจอได้ในมอ” เลยจำกัดตัวเองอยู่กับปัญหาในมหาลัย ที่จริงแล้ว real user มีเงื่อนไขแค่ 2 ข้อ คือ (1) เป็นคนที่ มีปัญหานั้นจริง และ (2) เรา ติดต่อเพื่อคุยด้วยได้ ไม่จำเป็นต้องเจอตัว โทร แชท ดีเอ็ม วิดีโอคอลก็นับ และไม่จำเป็นต้องอยู่ในมอ

นับเป็น real user

- คนใกล้ตัวที่อยู่ในกลุ่มจริง เช่น ยายที่กินยาหลายชนิด พ่อแม่หรือญาติที่ทำเกษตร ป้าที่ขายของออนไลน์
- ผู้ประกอบวิชาชีพ เช่น พยาบาล เภสัชกร ครู เจ้าของร้าน SME เกษตรกร คุยผ่าน LINE หรือโทรผ่านคนรู้จัก
- คนในคอมมูนิตี้ออนไลน์ที่โพสต์ปนเรื่องนั้น ใน Pantip Reddit หรือกลุ่ม Facebook ทัก DM ขอคุย 10 นาที
- ผู้ใช้ของแอปคู่แข่งที่รีวิว 1–2 ดาว เพราะเจอปัญหาเดียวกัน
- ตัวเราเอง ถ้าเราเป็นผู้ใช้จริงของปัญหา นับได้ 1 คน แต่ห้ามนับทั้งทีมเป็น 3

ไม่นับ

- “นักศึกษาทั่วไป” หรือ “คนไทยทุกคน” แบบลอย ๆ ที่ระบุตัวไม่ได้
- คนที่เรา เดา ว่าน่าจะมีปัญหา แต่ยังไม่เคยคุยด้วยจริง
- เอาสมาชิกทีม 3 คนมานับ ทั้งที่ไม่ได้เป็นผู้ใช้จริงของปัญหา

ถ้าไอเดียมาจากการค้นหา ปัญหาอ้างอิงจากอะไร นับยังไง

แยกหลักฐานเป็น 2 ชั้น อย่าเอามาปนกัน ชั้น A พิสูจน์ว่าปัญหามีจริง เราไม่ได้คิดเอง เช่น ในคอมมูนิตี้ Claude หรือ r/ClaudeAI หรือกลุ่ม Facebook มีคนพูดถึงปัญหาเดียวกัน ไม่ต้องทักก็ได้ แค่แคปหน้าจอมาแสดงว่ามีคนพูดถึงจริง รวมถึงโพสต์ที่มีคน react เยอะ (บอกจำนวน) รีวิวแอปคู่แข่ง และสถิติจาก data.go.th ทั้งหมดนี้ใช้อ้างอิงและทำ TAM แต่ไม่นับเป็น user · ชั้น B คือคนจริง 3 คนที่เราคุยด้วย ซึ่งคือสิ่งที่ Gate 1 ต้องการ ข่าวดีคือ พอค้นเจอคอมมูนิตี้แล้ว ถ้าอยากได้ครบ 3 คน ก็แค่ทักคนที่โพสต์เรื่องนั้น

ตัวอย่าง ไม่มีใครอยู่ในมอเลย แต่ผ่าน GATE 1 สบาย

ปัญหา ผู้สูงอายุกินยาหลายชนิดแล้วลืมหรือกินผิดเวลา · ผู้ใช้จริง 3 คน (ชั้น B) ยายของสมาชิกทีม เพื่อนบ้านสูงอายุ และพยาบาลที่ดูแลผู้ป่วยเรื้อรัง คุยผ่านญาติที่เป็นพยาบาล · อ้างอิงปัญหา (ชั้น A) กลุ่ม Facebook ดูแลผู้สูงอายุ มีโพสต์เรื่องนี้ 200+ คอมเมนต์ และสถิติผู้สูงอายุไทย 13 ล้านคนจาก data.go.th ใช้ทำ TAM

เกณฑ์ผ่าน SELF-CHECK

ทีมพร้อมเสนอหัวข้อ เมื่อ (ก) Scorecard ได้ ≥ 15/21 (ข) กรอก Problem Brief ครบทุกช่อง (ค) ระบุ “ผู้ใช้จริง 3 คน” ที่ติดต่อได้ก่อน Week 4 และ (ง) ผ่านทั้ง 3 ประตู non-negotiable ใน section 10 ถ้ายังไม่ถึง ให้ปีนบันไดปัญหาอีกขั้น แล้วลองใหม่

12 · GO FURTHER / ค้นคว้าต่อ

ไปค้นคว้าต่อที่ไหน

อ่านเรื่อง “เลือกปัญหา”

- “Fall in love with the problem, not the solution” โดย Uri Levine ผู้ก่อตั้ง Waze
- Paul Graham — “How to Get Startup Ideas” แนวคิด “live in the future, build what's missing”
- Y Combinator — Startup School ฟรี หัวข้อ evaluating startup ideas และ TAM
- Marty Cagan — Inspired product ที่ลูกค้ารักเริ่มจากปัญหา

ไปดูปัญหาจริงที่คน “บ่น”

- r/ClaudeAI, Anthropic Discord — คนอยากให้ AI ช่วยอะไร
- Product Hunt, Indie Hackers — ปัญหาที่มีคนยอมจ่าย
- Pantip หรือกลุ่ม Facebook เฉพาะกลุ่มเป้าหมาย ดูโพสต์ที่คน react เยอะ
- ข้อมูลเปิดภาครัฐที่ data.go.th ใช้ตัวเลขปัญหาระดับประเทศ ไว้ทำ napkin math

ประโยคปิดท้าย

เป้าหมายของเฟสนี้ไม่ใช่ “รีบมีไอเดีย” แต่คือ เข้าใจปัญหาลึกพอจนมั่นใจว่ามีคนอยากใช้ ปัญหาที่ดีคือ เล่าให้คนแปลกหน้าฟังแล้วเขาพยักหน้าว่า “ใช่เลย เราก็เจอ” ไปหาปัญหาแบบนั้นให้เจอ

ก้าวต่อไป แนวคิด AI SDLC

เมื่อได้หัวข้อที่ผ่านการวิเคราะห์ Root Cause และ TAM แล้ว ถึงเวลาเปลี่ยนบทบาทจาก “คนพิมพ์โค้ด (Coder)” เป็น “สถาปนิกระบบ (System Architect)” โดยใช้แนวคิด Agentic Workflow

Step 1: Milestone

แบ่ง Checkpoint ให้ชัดเจน

การทำงานกับ AI ต้องมีการแบ่งเฟสงาน เพื่อไม่ให้หลงทาง

- **Checkpoint 1 (Individual)** โฟกัสที่การพิสูจน์ Core Logic ให้นักศึกษาแยกกันทดสอบและพัฒนาฟีเจอร์หลักรายบุคคล เช่น ค้นคว้าโมเดล หรือเทสต์ Prompt เพื่อพิสูจน์ว่า AI ตอบโจทย์ Root Cause ได้จริง
- **Checkpoint 2 (Group)** นำ Core Logic ของทุกคนมาประกอบร่างกันเป็น Platform หรือ Product ที่สมบูรณ์

Step 2: Context

กำหนดบริบทของระบบ

สร้างไฟล์บริบท เช่น `architecture.md` ก่อนเริ่มโค้ด ระบุให้ชัดเจนว่ามีอะไรบ้าง

- เป้าหมายและปัญหาที่แก้ ดึงจาก 5 Whys
- Tech Stack ที่เลือกใช้ และโครงสร้าง Data Schema
- อธิบาย Requirement เป็นภาษาคนให้ละเอียดและเป็นเหตุเป็นผล ยิ่งชัด AI ยิ่งทำงานตรงใจ

Step 3: Agentic Loop

ลูปการพัฒนาแบบยุคใหม่

ฝึกใช้ AI IDE หรือ Terminal-based Agents เช่น Claude Code หรือ Cursor ด้วยกระบวนการนี้

- Prompt สั่งด้วยเป้าหมายที่ชัดเจนอิงจากไฟล์บริบท
- Generate ปล่อย AI สำรวจและสร้างโค้ด
- Review & Steer อ่านโค้ด ตรวจ Logic โยน Error Log กลับไปให้ AI หาทางแก้จนกว่าจะผ่าน

Step 4: Deploy

กลยุทธ์การทดสอบระบบจริง

เมื่อระบบเริ่มเป็นรูปเป็นร่าง สิ่งสำคัญคือการวางแผน deploy และทดสอบ

- โปรเจกต์ AI มักเจอข้อผิดพลาดจุกจิกถ้ารันผ่านระบบ CI/CD ปกติ
- ทางออก ถ้า deploy ไม่ขึ้น ให้เริ่มจากทางที่ “ทดสอบ product ได้เร็วที่สุด” ก่อน คือรันในเครื่อง localhost หรือใช้ free tier ของ hosting แล้วค่อยขยับไป hosting เต็มรูปแบบทีหลัง เพื่อโฟกัสกับตัว product

ทำไมต้อง AI SDLC

AI เขียนโค้ดได้เร็วกว่าคุณ แต่ AI ไม่รู้ว่าควรสร้างอะไร และไม่รู้ว่าสร้างไปทำไม การฝึกแบ่งเฟสงาน ตรวจสอบตรรกะ และทดสอบอย่างเป็นระบบ คือทักษะที่ Software Engineer ตัวจริงในยุคนี้ต้องมี

Starter Kit ตัวอย่างแนวทางของอาจารย์ จากปัญหา สู่ prototype

พอได้หัวข้อที่ GO แล้ว คำถามต่อไปคือ “จะเริ่มลงมืออย่างเป็นระบบยังไง” หัวข้อนี้ไม่ใช่ไฟล์ที่ต้องดาวน์โหลด แต่เป็น ตัวอย่าง ให้เห็นว่าอาจารย์ใช้ชุดเครื่องมือ ที่เรียกว่า Starter Kit V1 พาโปรเจกต์เดินจากปัญหาที่เลือกไว้ ไปเป็น requirements แล้ว design แล้ว prototype อย่างมีระเบียบ นักศึกษาไม่ต้องใช้ไฟล์นี้ ขอแค่เข้าใจ “โครงสร้าง” และแนวคิดเบื้องหลัง เพื่อเห็นภาพว่า SDLC ในยุค AI ทำงานกันแบบไหน

Starter Kit V1 ประกอบด้วยอะไรบ้าง

หัวใจของมันคือ “คลังเอกสาร (docs vault)” ที่จัดโฟลเดอร์ตามลำดับ SDLC บวกกับชุด agents และ skills ที่คอยดูแลให้เอกสารแต่ละชั้น “ตรงกัน” เสมอ

โฟลเดอร์ใน DOCS/	ทำหน้าที่อะไร
`01-requirements/` ความต้องการ	`backlog.md` คือรายการรวม FR/NFR ทั้งหมด เป็นแหล่งอ้างอิงหลัก · `01-spec/` เก็บรายละเอียดความต้องการ หนึ่งไฟล์ต่อหนึ่งหัวข้อ · `02-plan/` และ `03-task/` แบ่ง phase และแตกงานย่อยก่อนลงมือ dev
`02-design/` การออกแบบ	`DESIGN.md` คือ Design System หลัก ได้แก่ สี ตัวอักษร ระยะห่าง องค์ประกอบ UI · `01-prototypes/` เก็บ HTML mockup แบบมีวันที่และเวอร์ชัน · `02-technical/` เก็บ architecture, api-spec, db-spec, detailed-design, nfr-review · มี `feature-list.md` และ `user-journey.md` ประกอบ
`03-testing/` การทดสอบ	เก็บ acceptance-criteria แบบ Given–When–Then และ test-plan สำหรับตรวจว่าฟีเจอร์ตรงกับความต้องการ
`04-retrospectives/` · `05-log/` · `00-archived/`	สรุปบทเรียน · บันทึกงานรายวัน · เอกสารที่เลิกใช้ ตามลำดับ

นอกจากโฟลเดอร์เอกสารแล้ว ยังมีโฟลเดอร์ `.claude/` ที่บรรจุ agents เช่น requirement-writer, architecture-writer, prototype-writer, test-writer และ skills ที่เป็น “คำสั่ง” ให้เรียกใช้ เช่น `/capture-requirement` , `/build-prototype` , `/sync-technical-spec` ตัวช่วยเหล่านี้ทำหน้าที่สร้างและตรวจความสอดคล้องของเอกสารแต่ละชั้นให้ตรงกับชั้นก่อนหน้าโดยอัตโนมัติ

มี agents และ skills อะไรบ้าง

แต่ละ agent คือ “ผู้เชี่ยวชาญ” ที่รับผิดชอบเขียนหรือตรวจเอกสารหนึ่งชั้น ส่วน skill คือ “คำสั่ง” ที่เราพิมพ์เรียกใช้งาน จัดกลุ่มตามชั้นของ SDLC ได้ดังนี้

Agents (11 ตัว) — ผู้เขียนและตรวจเอกสาร

- ความต้องการ
  `requirement-writer` · `backlog-auditor`
- ฟีเจอร์และ journey
  `feature-journey-writer`
- ดีไซน์เชิงเทคนิค
  `architecture-writer` · `api-db-writer` · `detailed-design-writer` · `nfr-reviewer`
- Prototype
  `prototype-writer` · `prototype-auditor`
- ทดสอบและวางแผน
  `test-writer` · `phase-planner`

Skills (14 คำสั่ง) — สิ่งที่เราพิมพ์เรียก

- เริ่มจากความต้องการ
  `/capture-requirement`
- ตรวจความสอดคล้องทีละชั้น
  `/sync-feature-journey` · `/sync-architecture` · `/sync-api-db` · `/sync-detailed-design` · `/sync-technical-spec` · `/sync-test-plan` · `/sync-phase-plan`
- Prototype
  `/build-prototype`
- ตรวจทั้งสายงาน
  `/audit-backlog` · `/audit-pipeline`
- รวมหลายขั้นในคำสั่งเดียว
  `/run-requirements-phase` · `/run-technical-phase` · `/run-prototype-phase`

ลำดับการทำงาน จากปัญหา สู่ prototype

ปัญหา → spec → backlog.md → feature-list / user-journey → แตก 3 สาย → prototype

จาก backlog เอกสารจะแตกออกเป็น 3 สายขนานกัน คือ **technical** (architecture ⟶ api-spec/db-spec ⟶ detailed-design), **testing** (acceptance-criteria ⟶ test-plan) และ **prototype** (HTML mockup อิง DESIGN.md) แล้วจึงมารวมที่ phase plan ทั้งหมดยึดหลัก "แหล่งอ้างอิงเดียวต่อหนึ่งชั้น" และอ้างอิงกันด้วยรหัส FR/NFR เสมอ

แนวคิด SE ที่ซ่อนอยู่ใน STARTER KIT

- **ออกแบบก่อนโค้ด** ชั้น requirements และ design ราคาถูกและช่วยจับพลาดตั้งแต่เนิ่น ๆ
- **Logical design ไม่ผูกเครื่องมือ** architecture และ data model เขียนแบบภาษากลาง ยังไม่ระบุ database หรือ framework จนกว่าจะตัดสินใจจริง
- **ทุกอย่างสาวกลับถึงปัญหาได้** ด้วยรหัส FR/NFR และการอ้างอิงข้ามเอกสารแบบ wikilink
- **ทำทีละชั้น** ทำเสร็จหนึ่งชั้น หยุดดู แล้วค่อยไปต่อ ไม่สั่ง "ทำทั้งหมดทีเดียว"

เส้นด้ายเดียวกันตั้งแต่ต้นจนจบ

สังเกตว่า Problem จาก section 1–10 ต่อไปยัง Requirement/FR แล้ว Design แล้ว Prototype แล้ว Feature/Test **คือเรื่องเดียวกันทั้งหมด** นี่คือ Coherence ที่รูบริกให้คะแนน และ Starter Kit ก็ออกแบบมาให้เส้นด้ายนี้ไม่ขาด แม้นักศึกษาจะยังไม่ได้ใช้ไฟล์จริง แต่ให้จำ "รูปทรง" ของกระบวนการนี้ไว้ใช้กับโปรเจกต์ของตัวเอง

15 · SUBMIT / ส่งงาน

M1 ส่งอะไร และให้คะแนนยังไง

เมื่อทีมตัดสินใจหัวข้อได้แล้ว สิ่งที่ต้องส่งสำหรับ M1 คือ Team Charter หนึ่งหน้า ทีมละ 1 ไฟล์ ต่อไปนี้คือสิ่งที่ต้องมีในไฟล์ และเกณฑ์ที่อาจารย์ใช้ให้คะแนน

สิ่งที่ต้องส่ง

- **Team Charter 1 หน้า** (PDF หรือ Google Doc) ทีมละ 1 ไฟล์ ตัวแทน 1 คนแนบไฟล์แล้วกดส่งใน Google Classroom
- ในไฟล์มี ปัญหา · ผู้ใช้จริง 3 คน (Gate 1) · รูปร่าง MVP 3–5 FR (Gate 2) · การเข้าถึงข้อมูล (Gate 3) · SDLC ที่เลือก พร้อมเหตุผล
- ตั้งชื่อไฟล์ `M1-Charter_ชื่อทีม.pdf` · กำหนดส่งดูใน Google Classroom

เกณฑ์ให้คะแนน (เต็ม 6 คะแนน = 3% ของวิชา)

ให้คะแนนแต่ละข้อ 2 = ครบถ้วน · 1 = บางส่วน · 0 = ไม่มี

เกณฑ์	2	1	0
1. ปัญหาชัด และระบุผู้ใช้จริงได้	ชัด เจาะจงว่าใครคือผู้ใช้	ยังคลุมเครือ	ไม่มี
2. เข้าถึงผู้ใช้จริง ≥3 คน (ใครบ้าง + ติดต่อยังไง)	ระบุชื่อ + ติดต่อได้จริง	ระบุชื่อแต่ยังติดต่อไม่ได้	ไม่มี
3. เลือก SDLC พร้อมเหตุผล และ MVP อยู่ในกรอบ gate	มีเหตุผล + รูปร่างถูก	บอกเฉยๆ ไม่มีเหตุผล	ไม่มี

กฎการอนุมัติ

Charter ที่ยังไม่ผ่านประตูใดประตูหนึ่งใน 3 gate จะถูก **ส่งกลับให้แก้ ไม่ใช่ตก** และให้คะแนนครั้งเดียวเมื่อส่งฉบับแก้แล้ว เคล็ดลับ ให้คะแนน Scorecard ใน section 9 ได้ 15/21 ขึ้นไปก่อนส่ง

พิมพ์ หรือ บันทึกเป็น PDF